

TYPESCRIPT: INTERFACES & TYPE ALIASES

The Rulebooks of Code 🧱

Planrs Academy • Kids Coding Series

Messy Digital Library Challenge

Imagine you have a huge digital library, but all the books are mixed up and messy! How can we organize our digital world like a clean, neat cupboard? Today we will learn the tools that help us keep our code organized and tidy!

1. Quick Recap: Basic Building Blocks

Before moving forward, let's remember the simple building blocks of code. Basic types help TypeScript understand what kind of data we are working with:

- **string:** Stores text like "Arjun" or "Priya".
- **number:** Stores numbers like 11 or 12.
- **boolean:** Stores true/false values like true or false.

2. What is an Interface?

An **Interface** is like a blueprint or a rulebook for an object. Think of an identification card: every card must contain specific fields like a Name, Address, and Identification Number. If any of these are missing, the card is incomplete!

- **Interface = Rules** for what information must be present in an object.
- If a required property is missing or wrong, TypeScript will immediately alert you!

Code Example: Student Profile Rulebook

```
interface Student {  
  name: string;  
  age: number;  
  grade: number;  
}
```

This rulebook tells TypeScript exactly what properties every student object must have!

3. Code Comparison

✗ Messy Code (Errors!)

```
let student1 = {  
  nam: "Priya",      // Misspelled 'name'  
  age: "twelve"      // Wrong type (string  
                      // instead of number)  
}  
// Missing 'grade' property!
```

✓ Organized Code (Correct!)

```
let student1: Student = {  
  name: "Priya",  
  age: 12,  
  grade: 6  
}  
// Interface ensures everything is correct!
```

4. Mandatory vs. Optional Fields

Some information is mandatory, while other information can be optional:

- **Mandatory:** Fields like `studentName` must be provided.
- **Optional:** Fields like `emailID` can be skipped if not available. In TypeScript, we use a **question mark (?)** to mark optional fields!

```
interface SchoolForm {  
  studentName: string; // Required field  
  emailID?: string;    // Optional field (note the ?)  
}
```

5. What is a Type Alias?

A **Type Alias** is like a friendly nickname for a type! Instead of repeating a complex type definition over and over, we define a nickname once.

Basic Nickname Example:

```
type Pincode = string;
```

Whenever we need a pincode, we use our nickname `Pincode` instead of explaining it every time!

Union Types (Multiple Options):

A Type Alias with Union Types lets one nickname represent multiple possible types using the `|` (pipe) symbol:

```
type ID = number | string;
```

This means an ID can be either a number (like 12345) or a string (like "STU-001").

6. Working Together as a Team

Interfaces and Type Aliases complement each other to keep code structured and clean:

```
type ID = number | string;

interface CricketPlayer {
  name: string;
  runs: number;
  wickets?: number; // Optional field
  team: string;
  playerId: ID;      // Uses our Type Alias nickname!
}
```

7. When to Use Each One?

Tool	Purpose	Best Used For
Interfaces	Creates a blueprint / rulebook for data objects.	Defining structures of objects with properties and types.
Type Aliases	Creates nicknames for simple or combined types.	Simplifying complex names, Union types (number string).



Key Takeaways to Remember

- **Interfaces = Blueprints** for objects that enforce required properties.
- **Type Aliases = Nicknames** for types that make code cleaner and easier to read.
- **Optional Properties** use a ? symbol so they can be skipped if missing.
- Organizing code keeps it neat and error-free—just like organizing your school desk!

PRACTICE QUESTIONS FOR STUDENTS

Test Your Knowledge & Practice Coding! 

Answer the following questions to test your understanding of TypeScript Interfaces, Type Aliases, and object structures.

Q1 Multiple Choice

Which symbol is used in TypeScript to mark a property as **optional** in an interface?

- A) !
- B) ?
- C) *
- D) &

Q2 True or False

Statement: An Interface in TypeScript can be used as a nickname for a union type like `number | string`.

Q3 Spot the Error

Look at the interface and object declaration below. Why will TypeScript show an error?

```
interface Book {  
  title: string;  
  pages: number;  
  isAvailable: boolean;  
}  
  
let myBook: Book = {  
  title: "The Magic Treehouse",  
  pages: "120"  
};
```

Q4 Code Writing Practice

Create a Type Alias called `ResultStatus` that can be either the string `"PASS"` or the string `"FAIL"` using a Union Type.

Q5 Build Your Own Interface

Design an interface named `ZomatoOrder` for a food delivery item with the following requirements:

- `dishName` (string, required)
- `price` (number, required)
- `isVeg` (boolean, required)
- `specialInstructions` (string, **optional**)

Q6 Practical Application

Write a complete TypeScript object variable named `myOrder` that uses the `ZomatoOrder` interface you defined in Question 5.